

In this notebook, take a look at cell 18, I need the to provide structured JSON output, trying to create a response schema but how do I use the exact keys needed from the dict

Please walk me through what I am supposed to do, DO NOT make edits

Worked for 2m 36s >

Cell 18 is the structured-extraction code cell in [lab_4.ipynb \(line 689\)](#). I made no edits.

The key idea: `GOLD` has two levels of keys:

Python



```
GOLD.keys()  
# dict_keys(["L001", "L003", "L006"])
```

Those are letter IDs—not your response fields.

You need the keys from one of the inner dictionaries:

Python



```
FIELD_NAMES = list(next(iter(GOLD.values())).keys())
```

That produces:

Python



```
[  
    "applicant_name",  
    "amount_ghs",  
    "purpose",  
    "monthly_profit_ghs",  
    "has_collateral_or_guarantor",  
    "repayment_months",  
]
```



1. Define the types separately

1. Define the types separately

The keys can come directly from `GOLD`, but the JSON types should be explicitly defined because Python values alone do not reliably tell you nullability.

```
Python
FIELD_TYPES = {
    "applicant_name": {"type": "string"},
    "amount_ghs": {"type": "number"},
    "purpose": {"type": "string"},
    "monthly_profit_ghs": {"type": ["number", "null"]},
    "has_collateral_or_guarantor": {"type": "boolean"},
    "repayment_months": {"type": ["number", "null"]},
}
```

You can verify that you have neither omitted nor misspelled a key:

```
Python
assert set(FIELD_NAMES) == set(FIELD_TYPES)
```

2. Build the JSON Schema

Use the exact field names as the keys under `properties`:

```
Python
LOAN_SCHEMA = {
    "type": "object",
    "properties": {
        field_name: FIELD_TYPES[field_name]
        for field_name in FIELD_NAMES
    },
    "required": FIELD_NAMES,
    "additionalProperties": False,
}
```



Each part has a different job:

Each part has a different job:

- `properties` defines the permitted keys and their types.
- `required` says every key must appear.
- `additionalProperties: False` prevents the model from adding keys such as `"risk_level"`.
- `["number", "null"]` means the key must still appear, but its value may be `null`.

Strict Structured Outputs require all fields to be listed in `required`; nullable fields use a union with `null`. [🔗 Official OpenAI documentation](#)

Notice that `"required"` repeats the keys from `"properties"`. That is normal: `properties` describes fields; `required` controls whether they must be present.

3. Wrap it as a response format

For Chat Completions, the surrounding object would be:

```
Python
RESPONSE_FORMAT = {
    "type": "json_schema",
    "json_schema": {
        "name": "loan_application_extraction",
        "strict": True,
        "schema": LOAN_SCHEMA,
    },
}
```

You would then pass it into the completion call:

```
Python
response = client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    response_format=RESPONSE_FORMAT,
)
```

```

response = client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    temperature=0,
    max_tokens=500,
    response_format=RESPONSE_FORMAT,
)

```

Then parse the actual message content:

```

Python
content = response.choices[0].message.content
data = json.loads(content)

```

Your current `ask_llm()` helper does not accept or forward a `response_format` argument. Therefore, conceptually, you must either:

1. Make the API call directly inside `extract_fields()`, or
2. Later give `ask_llm()` an optional `response_format` parameter and forward it.

I have not made either change.

4. You still need the prompt requested by the assignment

Cell 18 explicitly requires an explicit schema in the prompt, one synthetic worked example, and the instruction not to guess. A response schema enforces structure, but it does not guarantee that the extracted facts are correct.

A clean prompt structure would be:

```

Plain text
Extract information from the loan application.

Return exactly these fields:
- applicant_name
- amount_obs

```



I have not made either change.

4. You still need the prompt requested by the assignment

Cell 18 explicitly requires an explicit schema in the prompt, one synthetic worked example, and the instruction not to guess. A response schema enforces structure, but it does not guarantee that the extracted facts are correct.

A clean prompt structure would be:

Plain text



Extract information from the loan application.

Return exactly these fields:

- applicant_name
- amount_ghs
- purpose
- monthly_profit_ghs
- has_collateral_or_guarantor
- repayment_months

If monthly profit or repayment duration is not stated, use null.
Do not infer or guess unstated facts.

Worked example:

[An example letter that is not one of the six assessed letters]

Example JSON:

[JSON containing all six exact keys]

Loan application:

[the actual letter]

Build the example as a Python dictionary and serialize it with `json.dumps()` :

Python



```
EXAMPLE_RESULT = {  
    "applicant_name": "Ama Asare",  
    "amount_ghs": 6000,
```



```
}
```

```
example_json = json.dumps(EXAMPLE_RESULT, indent=2)
```

That conversion matters:

- Python `True` becomes JSON `true` .
- Python `None` becomes JSON `null` .
- Python single-quoted dictionaries become valid double-quoted JSON.

5. Validate “exactly these keys”

JSON object order is not important, so compare sets:

Python



```
if set(data.keys()) != set(FIELD_NAMES):  
    print("Warning: response contains missing or unexpected keys")  
    return None
```

Do not compare `list(data.keys()) == FIELD_NAMES` unless you intentionally care about key order.

Important issue with your current Groq model

The notebook currently uses `llama-3.1-8b-instant` in [lab_4.ipynb \(line 105\)](#) . Groq documents that model as supporting JSON Object Mode, not strict JSON Schema Structured Outputs. Groq’s current strict-schema models are `openai/gpt-oss-20b` and `openai/gpt-oss-120b` . [🔥 Groq Structured Outputs documentation](#)

Also, Groq lists `llama-3.1-8b-instant` for shutdown on August 16, 2026—the current date—with `openai/gpt-oss-20b` as its replacement. [🔥 Groq deprecation notice](#)

Therefore:

- With your current model, use `response_format={"type": "json_object"}` plus the explicit prompt and manual key validation.
- If you use `openai/gpt-oss-20b` , the strict `json_schema` structure above is appropriate.
- Regardless of the mode, retain `json.loads` ↓ and graceful failure handling because the assignment explicitly requests them.

Almost. It covers the prompt-related techniques, but one requirement lives in the API call:

- Explicit schema and exact keys: yes.
- One worked example not taken from `LETTERS` : yes.
- "Do not guess": yes.
- Missing values use `null` : partially.
- `temperature=0` : yes, but only when passed to `ask_llm()` .
- JSON only, without Markdown fences: yes.

There is a small contradiction in the notebook: it says missing fields should use `null` , but declares only `monthly_profit_ghs` and `repayment_months` as nullable. Your prompt currently follows those declared types.

To match the notebook's wording more literally, change rule 2 to:

Plain text



If a field is not stated in the letter, use null. Do not guess.

However, your response schema must then allow `null` for every field. If you retain the notebook's specified types, keep your current narrower rule.

Also ensure the call contains:

Python



```
temperature=0
```

So yes—the proposed prompt plus `temperature=0` covers the intended techniques. Structured `response_format` is an additional safeguard, not a replacement for them.

3:21 PM

I need to make the LLM call in `extract_fields`?



Yes. `extract_fields(letter_text)` should handle the complete extraction process: